Resumen completo — Primer Parcial ICSW

Este documento junta, en un solo lugar y en orden de lectura, todo lo que entra en el primer parcial: Unidad 1 completa (incluye "No Silver Bullet"), Unidad 2 (Gestión de Producto, Requerimientos Ágiles, User Stories, Estimaciones Ágiles, Scrum) y Unidad 3 (SCM). Está pensado para poder leerse de punta a punta sin haber leído antes las notas de unidad por separado — los `[[wikilinks]]` marcan dónde hay más profundidad si hace falta, pero no son necesarios para seguir el hilo. Cada afirmación mantiene su fuente: `[Fuente: Apuntes]` (viene tal cual de la presentación de la cátedra), `[Fuente: Externa — autor, obra, página]` (viene de la bibliografía citada por la cátedra, verificada directamente contra el PDF original), o `[Fuente: Externa]` sin más detalle cuando hubo que investigar algo que ninguna presentación de la materia desarrolla.

Texto en este color = contenido que salió de resúmenes de compañeros y que **no pude confirmar** contra una fuente oficial de la cátedra ni contra la bibliografía citada — son solo 3 pasajes puntuales, tratalos como complemento no verificado, no como apunte firme.

UNIDAD 1 — Introducción a la Ingeniería de Software

1. ¿Qué es la Ingeniería de Software?

El término nació en 1968, en una conferencia de la OTAN convocada para discutir la **crisis del software**: los enfoques individuales de programación no escalaban a sistemas grandes y complejos, que resultaban poco confiables, más caros de lo esperado y con entregas tardías [Fuente: Apuntes; confirmado — Fuente: Externa, Sommerville cap. 1]. Software pasó a sostener literalmente la economía mundial: de las 10 empresas más valiosas del planeta hoy, la mayoría son compañías de software o profundamente dependientes de él (Apple, Microsoft, Alphabet, Amazon, NVIDIA), muy distinto al ranking de hace 20 años (petroleras, tabacaleras, bebidas) [Fuente: Apuntes].

Software, para la cátedra, es "un set de programas y la documentación que acompaña", con tres tipos básicos: de sistema, utilitarios y de aplicación [Fuente: Apuntes]. Sommerville coincide y agrega que también incluye la documentación de usuario y del sistema [Fuente: Externa]. Una idea complementaria: el software es conocimiento representado en distintos niveles de abstracción, desde la idea de negocio (más abstracta) hasta el hardware físico (más concreta), pasando por diseño, código fuente, bibliotecas/SO y código binario [Fuente: Apuntes].

La cátedra trae tres definiciones de **Ingeniería de Software** que coinciden en lo esencial: - **Parnas**: "construcción multipersona de software multiversión". - **IEEE/SWEBOK**: "la aplicación de un enfoque sistemático, disciplinado y cuantificable al desarrollo, operación y mantenimiento del software". - **Sommerville**: "una disciplina de la ingeniería que se ocupa de todos los aspectos de la producción de software, desde la concepción inicial del sistema hasta su operación y mantenimiento" [Fuente: Apuntes; verificado textual — Sommerville cap. 1.1.1].

Sommerville agrega un matiz clave: los ingenieros no buscan la perfección, sino resultados de calidad suficiente dentro de un plazo y presupuesto, aplicando teorías y métodos de forma pragmática, no dogmática [Fuente: Externa].

Por qué el software no es como una línea de producción industrial — cinco razones que trae la cátedra: es menos predecible, no hay "producción en masa" (casi ningún producto de software es igual a otro), no todas las fallas son errores, no se gasta con el uso, y no está gobernado por las leyes de la física [Fuente: Apuntes]. Esta idea conecta directamente con "No Silver Bullet" (sección 6).

2. Estado actual y antecedentes: la crisis del software

Pese a su importancia, los proyectos de software siguen fallando seguido. Problemas típicos: el producto final no satisface las necesidades reales del cliente, es difícil de extender/adaptar, tiene mala documentación, mala calidad, y termina costando y tardando más de lo presupuestado [Fuente: Apuntes]. La historia está llena de errores costosos: pérdida de la sonda Mariner I (1962, error de programación manuscrita), el gusano de Morris (1988), el error de cálculo del Pentium de Intel (1994), la explosión del cohete Ariane 5 (1996, desbordamiento al convertir un número de 64 a 16 bits), la pérdida del Mars Climate Orbiter (1999, mezcla de unidades imperiales y métricas), y fallas más recientes en frenos de Toyota o retiros de vehículos por fallas de software [Fuente: Apuntes].

Factores de éxito y fracaso (según encuestas de la industria que trae la cátedra) — cuando un proyecto sale bien, en orden: involucramiento del usuario, apoyo de la gerencia, enunciado claro de requerimientos, planeamiento adecuado, expectativas realistas, hitos intermedios, personas competentes. Cuando sale mal: requerimientos incompletos, falta de involucramiento del usuario, falta de recursos, expectativas poco realistas, falta de apoyo gerencial, requerimientos cambiantes [Fuente: Apuntes]. **Conclusión central de la unidad**: saber programar no alcanza — la mayoría de las causas de éxito/fracaso son de gestión y de personas, no técnicas.

3. Disciplinas que conforman la Ingeniería de Software

El **SWEBOK** (*Software Engineering Body of Knowledge*, IEEE, v3.0/2014) organiza el conocimiento en 15 áreas: requerimientos, diseño, construcción, pruebas, mantenimiento, gestión de configuración, gestión de ingeniería de software, proceso de ingeniería de software, modelos y métodos, calidad, práctica profesional, economía de la ingeniería de software, fundamentos de computación, fundamentos matemáticos, fundamentos de ingeniería [Fuente: Apuntes]. El **SEBoK** (*Systems Engineering Body of Knowledge*, v1.9.1/2018) es el equivalente para ingeniería de sistemas (paraguas más amplio: hardware + software + procesos), en 7 partes [Fuente: Apuntes]. ICSW cubre disciplinas técnicas (requerimientos, análisis y diseño, construcción, prueba, despliegue), de gestión (planificación, monitoreo y control de proyectos) y de soporte (gestión de configuración, aseguramiento de calidad, métricas) [Fuente: Apuntes].

4. Ejemplos de proyectos fallidos y exitosos

Ver la lista de errores costosos de la sección 2 como ejemplos de fracaso técnico/de gestión con consecuencias reales. Del lado del éxito, la encuesta de factores (sección 2) es la contracara: los proyectos exitosos comparten involucramiento del usuario y apoyo gerencial más que superioridad técnica pura.

5. Esencia y accidente: por qué “No Silver Bullet”

Frederick Brooks, en su ensayo **“No Silver Bullet — Essence and Accidents of Software Engineering”** (1986, agregado como capítulo 16 en la edición aniversario de *The Mythical Man-Month*), sostiene que ningún desarrollo tecnológico o de gestión, por sí solo, va a producir una mejora de un orden de magnitud (10x) en productividad, confiabilidad o simplicidad del desarrollo de software en una década [Fuente: Externa — Brooks, cap. 16, p. 177-203].

El argumento divide las dificultades en dos categorías (distinción aristotélica esencia/accidente): - **Esencia**: el trabajo intelectual de diseñar la estructura conceptual (qué datos, qué relaciones, qué comportamiento). No desaparece nunca. - **Accidente**: las dificultades que acompañan hoy la producción de software pero no son inherentes a él (sintaxis incómoda, falta de herramientas, compilación lenta). Sí se reducen con mejores herramientas.

Cuatro **propiedades irreducibles** de la esencia del software complejo [Fuente: Externa — Brooks, cap. 16]: - **Complejidad**: no hay dos partes iguales; escalar agrega tipos nuevos de elementos, no solo más de lo mismo. - **Conformidad**: debe ajustarse a interfaces e instituciones humanas arbitrarias, no a leyes naturales. - **Mutabilidad**: se modifica todo el tiempo porque es “pura materia de pensamiento”, infinitamente maleable. - **Invisibilidad**: no tiene representación geométrica natural; no se puede “dibujar” como un plano.

Saltos históricos que atacaron dificultades accidentales (nunca la esencia) — confirmados textualmente contra el capítulo [Fuente: Externa — Brooks, p. 186-188]: **lenguajes de alto nivel** (liberan al programa de gran parte de su complejidad accidental), **tiempo compartido** (preserva la inmediatez y permite mantener una visión general de la complejidad), **entornos de programación unificados** (Unix/Interlisp: bibliotecas integradas, formatos unificados, pipes y filtros que atacan la dificultad accidental de usar programas en conjunto).

“Esperanzas para la bala de plata” que Brooks descarta explícitamente, una por una [Fuente: Externa — Brooks, p. 188-196]: Ada y otros avances de lenguaje (“Ada no probará ser la bala de plata”), programación orientada a objetos (remueve dificultad accidental de expresión, pero “la complejidad del diseño en sí es esencial”), inteligencia artificial y sistemas expertos (distingue IA-1 de IA-2, veredicto: “no creo que la IA provea el quiebre”), “programación automática” (cita a Parnas: “siempre fue un eufemismo para programar con un lenguaje de más alto nivel”), programación gráfica (“nada convincente ha surgido todavía”), verificación de programas (“no creo que encontremos la magia acá”), entornos y herramientas (“el retorno de ahora en más será marginal”), estaciones de trabajo potentes (“no podemos esperar mejoras mágicas de ellas”).

Cuatro ataques prometedores a la esencia [Fuente: Externa — Brooks, p. 196-203]: 1. **Comprar en vez de construir** (buy versus build). 2. **Refinamiento de requerimientos y prototipado rápido**: “es realmente imposible que los clientes… especifiquen completa, precisa y correctamente los requerimientos exactos de un producto de software moderno antes de haber construido y probado algunas versiones”. 3. **Desarrollo incremental — crecer el software, no construirlo** (atribuido a Harlan Mills): “el secreto es que se cultiva, no se construye”. 4. **Grandes diseñadores**: la diferencia entre un diseño bueno y uno excelente es casi de un orden de magnitud y viene de la persona, no de la herramienta — Brooks propone 4 pasos

concretos para “cultivarlos”: identificarlos temprano, asignarles un mentor de carrera, un plan de desarrollo con aprendizajes concretos, y oportunidades de interacción entre pares.

Nota sobre esta unidad en el vault: la presentación de la cátedra retoma este marco con un dossier reinterpretado con herramientas de IA (aparece la marca de NotebookLM en los diagramas) y lo extiende explícitamente al desarrollo asistido por IA — la IA sigue atacando lo accidental (buena para producir el “cómo” una vez el problema está bien especificado), pero el cuello de botella sigue siendo humano: decidir con precisión el “qué” [Fuente: Apuntes — extensión de la cátedra, no parte del texto de 1986]. Ver [[No Hay Balas de Plata]] para el desarrollo completo de esta sección.

6. El resto de *El Mítico Hombre-Mes* (Brooks)

La cátedra desarrolla, en la presentación de Componentes de Proyecto, el resto de las ideas clásicas de Brooks sobre gestión de proyectos de software — todas del mismo libro. Ver [[Mítico Hombre-Mes (Brooks)]] para el detalle completo con ejemplos; en síntesis:

- **El multiplicador de costos**: un programa simple cuesta “1x”. Convertirlo en **producto de programación** (generalizado, probado, documentado) cuesta ~3x más. Convertirlo en **sistema de programación** (integrado con otros componentes) cuesta otro ~3x. Ambas cosas a la vez — lo que casi siempre se necesita en la práctica — cuesta ~**9x** el programa simple [Fuente: Apuntes].
- **Ley de Brooks**: agregar personal a un proyecto atrasado lo atrasa más — los nuevos necesitan mentoría (desvía foco de los que ya están) y más personas generan más interfaces de comunicación y más errores de integración.
- **El equipo quirúrgico (“El Cirujano”)**: el diseño de un sistema complejo debe fluir de una sola mente (o pocas), apoyada por roles complementarios (copiloto, administrador, editor, programadores de herramientas) — combina coherencia de diseño con productividad de varios ayudantes.
- **Integridad conceptual**: mejor un sistema que omita funciones poco frecuentes pero refleje un único conjunto de ideas de diseño coherente, que uno con muchas funciones buenas pero descoordinadas.
- **El efecto del segundo sistema**: el segundo sistema que se diseña es el más peligroso — el diseñador, envalentonado por el éxito del primero (deliberadamente simple), amontona todas las ideas guardadas, perdiendo foco y simplicidad.
- **El fracaso como problema combinatorio (Torre de Babel)**: el esfuerzo de comunicación crece según n(n-1)/2 — con 3 personas hay 3 líneas de comunicación, con 10 hay 45, con 50 hay 1225.
- **La representación es la esencia de la programación**: frente a datos desordenados, la solución no es código más denso sino rediseñar las estructuras de datos.
- **La Hipótesis Documental**: un pequeño conjunto de documentos clave (project workbook) funciona como radar del estado real del proyecto.

7. El proceso de software: definido vs. empírico

Un **proceso de software** es el conjunto de actividades, métodos, prácticas y herramientas que un equipo usa para transformar requerimientos, ideas y tiempo en productos y servicios de software [Fuente: Apuntes]. Sommerville confirma que las 4 actividades fundamentales de cualquier proceso son: especificación, diseño e implementación, validación y evolución [Fuente: Externa — Sommerville cap. 2].

```
- **Proceso definido**: asume que repetir la misma secuencia de pasos da siempre el mismo resultado (metáfora de la línea de producción); el control viene de seguir el proceso al pie de la letra.
```

- **Proceso empírico**: asume variables cambiantes — repetir el proceso puede dar resultados distintos; el control se logra con inspección frecuente y adaptación continua, siguiendo el ciclo **asumir** → **construir** → **retroalimentar** → **revisar** → **adaptar** [Fuente: Apuntes].

Esta distinción equivale, en terminología académica, a **dirigido por plan (plan-driven) vs. ágil** [Fuente: Externa — Sommerville cap. 2]. No existe un proceso "ideal": los sistemas críticos de seguridad suelen necesitar procesos definidos; los de requerimientos cambiantes se benefician de procesos empíricos/ágiles [Fuente: Externa]. Ver [[Proceso de Software]].

8. Ciclos de vida: definición y clasificación

Un **ciclo de vida** es la representación de un proceso de software: las fases por las que atraviesa el proyecto (o el producto) y el orden en que ocurren [Fuente: Apuntes]. Conviene distinguir el **ciclo de vida del producto** (desde la idea de negocio, sucesivas versiones, hasta el retiro) del **ciclo de vida del proyecto** (la porción de ese recorrido — típicamente inicial e intermedia — durante la cual se construye o actualiza una versión concreta) [Fuente: Apuntes].

La cátedra distingue tres tipos básicos: - **Secuencial**: las fases se ejecutan una después de la otra, sin volver atrás (ej. cascada). - **Iterativo/Incremental**: el sistema se construye y entrega en versiones sucesivas, cada una agregando funcionalidad. - **Recursivo**: mencionado, sin desarrollo propio en la presentación.

Sommerville agrupa los modelos genéricos en tres familias — cascada, desarrollo incremental, ingeniería orientada a la reutilización — y aclara que no son mutuamente excluyentes: en sistemas grandes suelen combinarse [Fuente: Externa — Sommerville cap. 2]. Ver [[Ciclo de Vida de Software]].

9. Ventajas, desventajas y criterios de elección de cada ciclo de vida

Este es un punto que ninguna presentación de la cátedra desarrolla — solo se remite a McConnell, *Rapid Development*, cap. 7, como ampliación. Se verificó directamente contra ese capítulo (páginas 133-162) [Fuente: Externa — McConnell, *Rapid Development*, cap. 7].

Los 10 modelos que desarrolla McConnell, con una descripción breve de cada uno:

1. **Cascada pura**: el modelo lineal clásico — requerimientos → diseño → implementación → prueba → mantenimiento, cada fase se completa por entero antes de pasar a la siguiente.
2. **Code-and-Fix**: sin especificación ni diseño formal; se escribe código directamente y se corrige sobre la marcha.
3. **Espiral**: ciclos repetidos de análisis de riesgo → prototipado → desarrollo → evaluación, reduciendo incertidumbre en cada vuelta.
4. **Cascadas modificadas**: variantes de la cascada pura con retroalimentación entre fases adyacentes, superposición de fases, o subproyectos paralelos.

5. **Prototipado evolutivo**: se construye un prototipo real (no xdescartable) y se refina con feedback del usuario hasta ser el producto final.
6. **Entrega por etapas**: el diseño completo se hace por adelantado; la construcción y entrega se dividen en etapas sucesivas.
7. **Diseño a cronograma**: la fecha es fija e innegociable; las funcionalidades se priorizan y agregan hasta que se acaba el tiempo.
8. **Entrega evolutiva**: combina entrega por etapas con prototipado evolutivo — primero el núcleo del sistema (funciones de bajo nivel poco propensas a cambiar), después el resto evoluciona con feedback [Fuente: Externa — McConnell, p. 152].
9. **Diseño a herramientas**: solo se incluye la funcionalidad que las herramientas/librerías existentes soportan directamente [Fuente: Externa — McConnell, p. 152-153].
10. **Software comercial listo para usar (COTS)**: en vez de construir, se evalúa comprar/adoptar un producto ya existente [Fuente: Externa — McConnell, p. 153].

Tabla 7-1 de McConnell — "Fortalezas y debilidades de cada modelo" (verificada, p. 156-157; cada celda es la calificación del libro: Pobre / Aceptable / Excelente):

Criterio	Cascada pura	Code-and-Fix	Espiral	Casc. modificadas	Prototip. evolutivo	Entrega por etapas	Entrega evolutiva	Diseño a cronograma	Diseño a herramientas	COTS
Funciona con requerimientos poco entendidos	Pobre	Pobre	Excelente	Acept.-Exc.	Excelente	Pobre	Acept.-Exc.	Pobre-Acept.	Aceptable	Excelente
Funciona con arquitectura poco entendida	Pobre	Pobre	Excelente	Acept.-Exc.	Pobre-Acept.	Pobre	Pobre	Pobre	Pobre-Exc.	Pobre-Exc.
Produce sistema confiable	Excelente	Pobre	Excelente	Excelente	Aceptable	Excelente	Acept.-Exc.	Aceptable	Pobre-Exc.	Pobre-Exc.
Produce sistema con gran margen de crecimiento	Excelente	Pobre-Acept.	Excelente	Excelente	Excelente	Excelente	Excelente	Acept.-Exc.	Pobre	N/A
Gestiona riesgos	Pobre	Pobre	Excelente	Aceptable	Aceptable	Aceptable	Aceptable	Acept.-Exc.	Pobre-Acept.	N/A
Se ajusta a un cronograma predefinido	Aceptable	Pobre	Aceptable	Aceptable	Pobre	Aceptable	Aceptable	Excelente	Excelente	Excelente
Tiene bajo overhead	Pobre	Excelente	Aceptable	Excelente	Aceptable	Aceptable	Aceptable	Aceptable	Acept.-Exc.	Excelente
Permite correcciones a mitad de camino	Pobre	Pobre-Exc.	Aceptable	Aceptable	Excelente	Pobre	Acept.-Exc.	Pobre-Acept.	Excelente	Pobre
Da visibilidad de progreso al cliente	Pobre	Aceptable	Excelente	Aceptable	Excelente	Aceptable	Excelente	Aceptable	Excelente	N/A
Da visibilidad de progreso a la gerencia	Aceptable	Pobre	Excelente	Acept.-Exc.	Aceptable	Excelente	Excelente	Excelente	Excelente	N/A

Criterio	Cascada pura	Code-and-Fix	Espiral	Casc. modificadas	Prototip. evolutivo	Entrega por etapas	Entrega evolutiva	Diseño a cronograma	Diseño a herramientas	COTS
Requiere poca sofisticación del equipo	Aceptable	Excelente	Pobre	Pobre-Acept.	Pobre	Aceptable	Aceptable	Pobre	Aceptable	Aceptable

Las 10 preguntas de McConnell para elegir el ciclo de vida más rápido para un proyecto (sección 7.11, p. 154-156) — esto responde directamente al criterio de elección del temario: 1. ¿Qué tan bien entienden cliente y equipo los requerimientos al inicio? ¿Es probable que cambien mucho en el camino? 2. ¿Qué tan bien se entiende la arquitectura del sistema? ¿Habrá que hacer cambios arquitectónicos grandes a mitad de proyecto? 3. ¿Cuánta confiabilidad se necesita? 4. ¿Cuánto hay que planificar/diseñar por adelantado pensando en versiones futuras? 5. ¿Cuánto riesgo implica el proyecto? 6. ¿El proyecto está limitado a un cronograma predefinido? 7. ¿Hace falta poder hacer correcciones a mitad de camino? 8. ¿Hace falta darle al cliente visibilidad del progreso durante todo el proyecto? 9. ¿Hace falta darle a la gerencia esa misma visibilidad? 10. ¿Cuánta sofisticación del equipo requiere usar exitosamente ese modelo?

Regla práctica de síntesis: requerimientos claros + baja incertidumbre + sin apuro del cliente → conviene un modelo secuencial (cascada); cliente quiere resultados rápido con las funciones principales → conviene iterativo-incremental (entrega por etapas/evolutiva); el objetivo es bajar riesgo en sistemas grandes o muy inciertos → conviene espiral/recursivo.

10. Qué es un proyecto de software

Cuatro características que lo definen [Fuente: Apuntes]: - **Es único**: siempre hay algo que lo distingue de otro proyecto parecido. - **Duración limitada**: es temporario — termina cuando se alcanzan los objetivos. - **Orientado a objetivos**: existe para lograr resultados claros, no ambiguos, y alcanzables. - **Tareas interrelacionadas basadas en esfuerzos y recursos**: la complejidad de un proyecto es sistémica, no la suma de tareas independientes.

11. Administración de proyectos y la Restricción Triple

Administrar un proyecto es “tener el trabajo hecho en tiempo, con el presupuesto acordado y habiendo satisfecho las especificaciones” [Fuente: Apuntes]. El marco central para balancear decisiones es la **Restricción Triple**: alcance, tiempo y costo son las tres variables que compiten entre sí; la calidad no es un cuarto vértice, sino el resultado del balance entre las otras tres [Fuente: Apuntes]. Ver [[Restricción Triple]].

El **líder de proyecto** media entre el cliente/subcontratistas/reguladores por un lado y la gerencia/gerentes funcionales por el otro. Un **equipo de proyecto** es un grupo chico, con conocimientos diversos, que logra sinergia y responsabilidad compartida [Fuente: Apuntes]. El **plan de proyecto** es “lo que una hoja de ruta es a un viaje”: responde qué se hace, cuándo, cómo y quién.

12. Planificación de un proyecto de software (8 elementos)

1. **Definición del alcance**: alcance del *producto* (todas las características posibles) vs. alcance del *proyecto* (todo el trabajo, y solo ese trabajo, necesario para entregarlo). El primero se mide contra la especificación de requerimientos; el segundo contra el plan de proyecto.
2. **Definición de proceso y ciclo de vida** (ver secciones 7-9).
3. **Estimación**: tamaño, esfuerzo, calendario, costo, recursos críticos. Ver Unidad 2 para estimación ágil, y más abajo para estimación tradicional.
4. **Gestión de riesgos**: identificar → analizar → planificar respuesta → seguimiento y control → aprendizaje, en ciclo continuo a lo largo de todo el proyecto. Ver [[Gestión de Riesgos de Proyecto]]. 5-6. **Asignación de recursos** y **programación de proyectos**.
5. **Definición de controles**: monitorear avance real contra lo planificado.
6. **Definición de métricas**: proceso, proyecto y producto — tamaño, esfuerzo, tiempo, defectos [Fuente: Apuntes]. “Manténgalo simple”: una métrica solo vale la pena si aporta información nueva de beneficio práctico.

13. Monitoreo, control y factores de éxito/fracaso

Monitorear y controlar significa comparar constantemente lo planificado contra lo real [Fuente: Apuntes]. Los tres factores más determinantes para el éxito: monitoreo y feedback constante, misión/objetivo claro, comunicación. Causas de fracaso: fallas al definir el problema, planificar con datos insuficientes, planificación hecha solo por “el grupo de planificaciones” sin involucrar a quien ejecuta, falta de seguimiento del plan, planes pobres en detalle, planificación de recursos inadecuada, estimaciones basadas en supuestos sin datos históricos, y que nadie esté a cargo [Fuente: Apuntes].

14. Vínculo proceso–proyecto–producto

El vínculo central de toda la unidad: un **proceso** de software se instancia y se adapta en un **proyecto** concreto; a ese proyecto se incorporan **personas** y se automatiza con **herramientas**; el proyecto, como resultado, obtiene un **producto** [Fuente: Apuntes]. El proceso es la receta general, el proyecto es su ejecución concreta con fecha de inicio y fin, y el producto es lo que ese proyecto entrega.

UNIDAD 2

15. Gestión de Producto

Crear un producto de software no termina en “escribir código que funciona”: hay una cadena completa desde validar una hipótesis hasta lograr crecimiento y lealtad del usuario [Fuente: Apuntes]. Mensaje central (pensamiento Lean): “el mayor riesgo no es fallar técnicamente, es construir a la perfección algo que nadie quiere”. Estadística de la industria: 45% de las características desarrolladas nunca se usan, 19% raramente, 16% a veces, 13% a menudo, 7% siempre [Fuente: Apuntes] (coincide con el 64% “rara vez o nunca” que cita Jim Johnson/Standish Group 2002, visto también en User Stories).

Ciclo Lean Startup — Construir → Medir → Aprender [Fuente: Externa — Eric Ries]. Posponer el lanzamiento para evitar mostrar “cero ingresos, cero clientes” (la “audacia del cero”) garantiza aislamiento

y aumenta el riesgo de fracaso masivo; el antídoto es iterar el ciclo continuamente. Ver [[Lean Startup]].

MVP (Producto Mínimo Viable): “un experimento diseñado para confrontar la realidad lo antes posible. Requiere simplificación y evitar la sobre-construcción. En caso de duda, simplifique” [Fuente: Apuntes]. Contrasta la “audacia de cero” contra “el costo del retraso” (posponer el MVP pierde feedback esencial). Un MVP no es “lo que quedó a medio hacer” — es la porción más chica y barata elegida a propósito para probar una hipótesis específica. Ver [[MVP]].

Casos Dropbox y Facebook: un riesgo técnico se prueba con una prueba de humo sin código (Dropbox, video demo); un riesgo de comportamiento humano se prueba con tracción real medida (Facebook). Dos “tracks” de validación distintos: **Hipótesis de Valor** (¿entrega valor? se mide con retención) y **Hipótesis de Crecimiento** (¿cómo lo descubren usuarios nuevos? se mide con NPS/referencias) — un producto puede tener alta retención y bajo crecimiento, o viceversa.

UVP y table stakes: el MVP debe enfocarse en la Propuesta de Valor Única (la hipótesis principal), ofreciendo lo mínimo indispensable en este orden de prioridad: **Funcionalidad** → **Fiabilidad** → **Usabilidad** → **Diseño**. La base (funcionalidad) debe estar completa y cohesiva, aunque el producto sea limitado en capas superiores. Regla práctica: si los usuarios piden requerimientos totalmente distintos entre sí, no se encontró mercado; si piden todos la misma característica, es señal de pivotar.

El pivote: Fase A (se lanza el MVP con una UVP), Fase B — señal dispersa (“es genial, pero necesito X”, distinto para cada uno — no hay mercado), Fase C — señal clara (las respuestas convergen → se construye un MVP2/pivote sin abandonar la UVP original).

Matriz de priorización (estructura idéntica a la matriz de Eisenhower, urgente/importante): **Hacer** (urgente+importante → entra al MVP), **Planear** (no urgente+importante → Beta), **Delegar** (urgente+no importante → integraciones de terceros/APIs), **Eliminar** (ninguna de las dos → fuera del roadmap).

Escalando después del encaje — MMF, MVF y “corte geológico”: **MMF** (*Minimum Marketable Feature*, alta certeza comercial: pieza más pequeña de funcionalidad liberable con valor inmediato) [Fuente: Externa — Denne y Cleland-Huang, *Software by Numbers*, 2004] y **MVF** (*Minimum Viable Feature*, alta incertidumbre: una mini-MVP dentro de un producto ya existente — término de la cátedra, no ampliamente documentado fuera de este contexto [Fuente: Apuntes, sin verificación externa]). Aunque MVP/MMF/MVF son “atómicos” de cara al negocio, por dentro se cortan en User Stories (rebanadas de ejecución técnica).

Cronología del lanzamiento: **MVP** (Fase 1, aprendizaje: “¿vale la pena construirlo?”) → **MMP/MMR1** (Fase 2, lanzamiento: *Minimum Marketable Product*, primer release a *early adopters* con las MMFs esenciales: “¿está listo para vender?”) → **MMR2…MMRn** (Fase 3, evolución: incrementos sucesivos).

MLP (Minimum Lovable Product — Producto Mínimo Adorable) [Fuente: Externa — Brian de Haaff, 2013, libro *Lovability*, 2017]: evolución MVP (funcional) → MMP (comercializable) → MLP (memorable), para mercados maduros donde “viable” ya no alcanza. Motivos: reputación de marca, diferenciación, bucle de retroalimentación leal. **Los 10 bloques de construcción de la adorabilidad** (de Haaff): Esperanza, Satisfacción, Cuidado (Utilidad) → Confianza en uno mismo, Confianza en el sistema, Escala, Sostenibilidad (Crecimiento) → Motivación, Diversión (Inspiración) → Halo. Ejemplos que llegan a “Halo”: Spotify (listas personalizadas), Instagram (filtros creativos), WhatsApp (fiabilidad y simplicidad radical).

Matriz comparativa final:

	MVP (Viable)	MMP (Comercializable)	MLP (Adorable)
Objetivo	Validar concepto económicamente	Atender base amplia con funcionalidad clave	Crear conexión emocional
Enfoque	Investigar y empatizar	Construir y refinar	Diseñar para el deleite
Beneficio	Informar el alcance	Asegurar calidad y salida al mercado	Aumentar lealtad
Ejemplo	Entrevistas y videos	Lanzamiento Beta	App con animaciones personalizadas

Nota de compañeros (no verificada, no incluida como afirmación de cátedra): un resumen de compañero trae un framework alternativo "Preparar un MVP" de 6 pasos (Encontrar un Nicho → Crear Roadmap Realista → Investigar Competencia → Pre-vender el MVP → Testear suposiciones → Focalizar en funcionalidades principales) — puede servir como checklist práctico adicional, pero no proviene de la presentación oficial ni fue verificado contra una fuente citada.

16. Requerimientos Ágiles

Ágil **no es una metodología ni un proceso**: es una ideología con principios que guían el desarrollo, resumida en el **Manifiesto Ágil (2001)**: 4 valores — individuos e interacciones sobre procesos y herramientas, software funcionando sobre documentación extensiva, colaborar con el cliente sobre negociación contractual, responder al cambio sobre seguir un plan — y 12 principios [Fuente: Apuntes]. "Un compromiso útil entre nada de proceso y demasiado proceso" (Fowler, 2001). Ser ágil no es ser indisciplinado: hay un punto medio ("Good Agile") entre no planificar nada y planificar todo de antemano.

Cinco niveles de requerimiento (ejemplo bancario): Negocio ("disminuir X% de tiempo en procesos manuales") → Usuario ("consultar estado de cuenta en línea") → Funcional ("generar reporte de saldos") → No funcional ("formato PDF, seguridad según ley bancaria") → Implementación ("servidores en la nube") [Fuente: Apuntes]. Se agrupan en una pirámide: Negocio+Usuario = **Dominio del Problema**; Funcional+NoFuncional+Implementación = **Dominio de la Solución**.

Tradicional vs. ágil (dos triángulos): tradicional/dirigido por plan (Requisitos fijos, Recursos y Tiempo se estiman) vs. ágil/dirigido por valor (Recursos y Tiempo fijos, Alcance se estima y prioriza por valor — triángulo invertido) [Fuente: Apuntes]. Coincide con la Gestión Ágil de Requerimientos de Scott Ambler (2004): pila priorizada por valor, cada iteración implementa lo de mayor prioridad, reordenable en cualquier momento. "Los requisitos cambiantes son una ventaja competitiva" — no un problema, como en el enfoque BRUF (*Big Requirements Up Front*, donde el 64% de funcionalidades especificadas de antemano terminan usándose "rara vez o nunca", Standish Group 2002).

17. User Stories

Las 3 C's (Ron Jeffries, 2001) [Fuente: Externa — Cohn, cap. 1]: - **Tarjeta (Card)**: la frase verbal escrita. "Lo que se escribe en la tarjeta no es importante, lo que uno habla, sí" (Jeff Patton). - **Conversación**: quién realiza la acción, qué hace el sistema — donde se negocian los detalles. - **Confirmación**: por qué es necesaria la actividad, se traduce en criterios de aceptación y pruebas.

Las historias son multipropósito (Kent Beck, *Extreme Programming Explained*, 1999): necesidad del usuario, descripción del producto, ítem de planificación, "token" para conversación, mecanismo para

diferir conversación.

Formato: *"Como* `<rol>`*, yo puedo* `<actividad>` *de forma tal que* `<valor>`*"*. Ejemplo (GPS): "Como Conductor quiero buscar un destino a partir de sus coordenadas para conocer el camino a recorrer." Criterios de aceptación: formato de coordenadas + orientación.

INVEST (Bill Wake, 2003) — seis características de una buena historia: **I**ndependiente, **N**egociable, **V**aliosa, **E**stimable, **S**mall (chica), **T**esteable. Ver [[INVEST]]. Para **Independencia**: si dos historias quedan dependientes (ej. tres historias para Visa/MasterCard/Amex), conviene combinarlas o buscar otro eje de división. Para **Negociabilidad**: diferencia entre una nota-recordatorio ("Aceptar Visa, MC y Amex. Considerar Discover.") y sobrecargar de detalle decidido de antemano — ese detalle debe vivir en los tests, no en el frente de la tarjeta. Ejemplo de historia que viola **Valuable**: "El sistema utilizará Log4J para registrar errores" — al usuario no le importa la librería, solo que los errores queden registrados.

Descomposición: **Epic** (historia grande, no lista para una iteración) → **User Story** (funcionalidad concreta) → **Theme** (colección de historias relacionadas). Las historias deben ser **porciones verticales** (UI + lógica + persistencia de una sub-funcionalidad completa), no horizontales (solo UI, o solo BD) — lo vertical entrega valor demostrable de punta a punta. **Historia compuesta** (agrupa sub-funcionalidades, se divide por ellas) vs. **historia compleja** (grande por incertidumbre, se divide en un [[Spike]] + historias de desarrollo real). Historias muy chicas (bugs, cambios visuales) conviene combinarlas.

Modelado de roles: cuando no hay acceso a usuarios reales, se modelan roles antes de escribir historias — **Brainstorm** (tarjetas sin discusión, ~15 min) → **Organizar** (superponer las parecidas) → **Consolidar** (fusionar o descartar) → **Refinar** (agregar atributos: frecuencia de uso, expertise, objetivo). Técnicas adicionales: **Personas** (nombre, foto, descripción concreta para un rol importante) y **Personajes extremos** (Papa, traficante de drogas — ayuda a encontrar historias que pasarían desapercibidas).

Usuarios proxy (cuando no hay usuarios reales): gerentes, marketing, vendedores, expertos del dominio, clientes (quien decide la compra, no necesariamente quien usa), capacitadores/soporte — "no son ideales… EVÍTELOS". Cada uno tiene un sesgo característico (marketing conoce el mercado, no al usuario; ex-usuarios pueden desalinearse con el tiempo; clientes priorizan seguridad más que el usuario; capacitadores priorizan facilidad de enseñar). Conviene usar más de un tipo a la vez, o liberar versiones tempranas para feedback real.

Criterios de aceptación: definen los límites de la historia (cuándo dejar de agregar funcionalidad), deben describir una intención (no una solución) y ser independientes de la implementación. Ver [[Criterios de Aceptación]]. Las **pruebas de aceptación** se escriben antes de programar, porque obligan a explicitar supuestos. Ejemplo completo (tarjeta de crédito): criterio "Se acepta Visa, MC y Amex. En compras >$100 se pide el número del dorso" → pruebas: Visa (pasa), Diner's Club (falla), monto <$100, monto >$100.

Spikes: historia especial para sacarle riesgo/incertidumbre a otra historia, mediante investigación o prototipado con tiempo limitado; técnicos (dominio de solución) o funcionales (interacción con usuario); usar como excepción, no regla. Ver [[Spike]].

Definition of Ready / Definition of Done: no desarrolladas en las diapositivas de la cátedra — presentadas como práctica estándar de la industria [Fuente: Externa]. **DoR**: condiciones antes de tomar

una historia para una iteración (típicamente, que cumpla INVEST). **DoD**: condiciones para considerarla terminada (código escrito, revisado, testeado e integrado). Ambos conectan con **Estimable** y **Testable** de INVEST.

Qué NO son las User Stories [Fuente: Externa — Cohn, cap. 12]: no son especificaciones detalladas como los casos de uso; son expresiones de intención; no se detallan al principio del proyecto (se difiere el análisis hasta justo antes de empezar); necesitan poco mantenimiento y pueden descartarse; sirven de entrada a la documentación incremental, no son la documentación final.

Ejercicio "qué huele mal" (Cohn, Apéndice B): las que "huelen mal" fijan tecnología de implementación de antemano (Log4J, librería de gráficos) o son en realidad restricciones de proyecto (fecha de entrega), en vez de describir la necesidad del usuario.

Priorización del backlog: las de prioridad alta se modelan con más detalle (van a implementarse pronto); las de baja, con menos (todavía pueden cambiar).

18. Estimaciones Ágiles

"No ser dogmático sobre nada — ser pragmático es la clave del éxito en casi todo" [Fuente: Apuntes]. Si las estimaciones se usan como **compromisos**, son peligrosas; lo más beneficioso de estimar no es el número final sino el proceso (la conversación que obliga a tener).

Story Points (SP): medida de **tamaño relativo**, no de tiempo. Ver [[Story Point]]. Las personas estiman mal en absoluto pero bien comparando (ejercicio de "dar tamaño" a perros por foto: más fácil decir "el doble de grande" que estimar centímetros exactos). **Tamaño vs. Esfuerzo**: tamaño indica complejidad + trabajo + magnitud de una historia; esfuerzo (tiempo) varía según habilidad/experiencia de quien la hace — "**tamaño NO ES esfuerzo**".

Escalas: números 1-10, talles de remera (S/M/L/XL/XXL), potencias de 2, Fibonacci — una vez elegida, no se cambia (es el "metro patrón" del equipo). **Ejercicio de las burbujas** (Rachel Weston, CST): tres círculos superpuestos por historia — Complejidad, Esfuerzo, Duda — que se traducen a un talle (M, M, XL) y después a un número Fibonacci (5, 5, 13).

Velocidad (Velocity): métrica de progreso — suma de story points de historias **completadas** (no parciales) en la iteración. Ver [[Velocidad (Velocity)]]. Función central: **corrige errores de estimación** sin re-estimar nada — si un equipo estimó un proyecto en 200 puntos creyendo completar 25/iteración (8 iteraciones) pero su velocidad real es 20, automáticamente sabe que tomará 10 iteraciones, no 8.

Planning Poker [Fuente: Externa — Cohn, cap. 6; origen: James Grenning, 2002]: combina opinión de experto, analogía y desagregación. Solo estiman los desarrolladores (el PO responde preguntas, no estima) — "las personas más competentes en resolver una tarea deben ser quienes las estiman" (Jørgensen, 2004). Mecánica: se elige una **base story** (**historia canónica**) de referencia → se lee la historia → cada estimador elige carta en privado → se dan vuelta todas juntas (evita anclaje) → si coinciden, listo; si no, discuten (especialmente el más alto y el más bajo) y repiten. Ver [[Planning Poker]]. Origen: evolución lúdica del **Wideband Delphi** de Barry Boehm (años 70-80), que reduce el error de estimación ~40% respecto de promediar estimaciones individuales [Fuente: Externa — McConnell, sec. 13.2].

Estimación tradicional (no ágil) — presentación “07 Estimaciones de Sw” [Fuente: Apuntes]: por definición, una estimación no es precisa; estimar no es planear y planear no es estimar; las estimaciones no son compromisos. **Métodos**: basados en la experiencia (datos históricos, juicio experto puro o Delphi/Wideband Delphi, analogía), basados exclusivamente en recursos, basados en el mercado, basados en componentes del producto/proceso, métodos algorítmicos. **Wideband Delphi**: se dan especificaciones a un grupo de expertos → se reúnen para discutir producto y estimación → cada uno estima en privado y envía al coordinador → se revisan estimaciones propias y ajenas (anónimas) → se repite hasta converger. **Fórmula PERT** para estructurar el juicio de experto: (optimista + 4×habitual + pesimista) / 6 — “mucho cuidado con esto, es un buen comienzo pero un pésimo final”. **Fuente más común de error**: omitir actividades necesarias (requerimientos faltantes, actividades de desarrollo faltantes como documentación/revisiones/testing de datos, actividades generales como licencias/reuniones) — de ahí el uso de **buffers** (contingencia). Cita de cierre: “Nunca tenga temor que estimaciones creadas por desarrolladores sean demasiado pesimistas, dado que los desarrolladores siempre generan cronogramas demasiado optimistas” (Chris Peters, Microsoft VP).

Tabla comparativa tradicional vs. ágil (verificada, consistente con lo ya resuelto):

Aspecto	Ambiente Tradicional	Ambiente Ágil
Quién estima	Gerente de proyecto / experto asignado	El equipo de desarrollo completo
Unidad de medida	Tiempo (horas/días), costo	Tamaño relativo (story points)
Precisión buscada	Alta, detallada por tarea	Suficiente para planificar, no exacta
Momento de estimar	Mayormente al inicio (planificación)	Continuo, iteración a iteración
Naturaleza del compromiso	Se trata como compromiso/promesa	Explícitamente no es un compromiso
Manejo de incertidumbre	Buffers/contingencia explícitos	Se absorbe reestimando con velocidad real

19. Framework Scrum

Scrum: “un marco de trabajo liviano que ayuda a las personas, equipos y organizaciones a generar valor a través de soluciones adaptativas para problemas complejos” [Fuente: Externa — *The Scrum Guide*, 2020]. Basado en **proceso de control empírico**, tres pilares: **Transparencia** (revelar la realidad tal como es), **Inspección** (detectar variaciones frecuentemente), **Adaptación** (ajustar de inmediato — inspección sin adaptación es desperdicio). Cinco **valores**: coraje, foco, compromiso, respeto, apertura.

Roles (accountabilities): **Product Owner** (el “qué” y “por qué” — maximiza valor, gestiona el Product Backlog; no es “un escriba de tickets”), **Scrum Master** (la efectividad — líder servicial, agente de cambio, remueve impedimentos; no es “un secretario ni administrador de Jira”), **Developers** (el “cómo” y la calidad — crean el incremento, gestionan el Sprint Backlog, se autogestionan).

Eventos (todos con timebox fijo, máximos para un Sprint de 1 mes): **Sprint** (contenedor, ≤1 mes, “seguro para fallar”; Sprint Goal inamovible, alcance negociable si hay aprendizaje); **Sprint Planning** (≤8h: por qué/qué/cómo); **Daily Scrum** (15 min: re-planificación de Developers para Developers, no un reporte de estado); **Sprint Review** (≤4h: sesión colaborativa, inspección del incremento con stakeholders); **Sprint Retrospective** (≤3h: mejora de personas/proceso/herramientas); **Refinamiento del Product Backlog** (continuo, ~10% del tiempo del Sprint).

Artefactos y compromisos: Product Backlog → **Product Goal**; Sprint Backlog → **Sprint Goal**; Increment → **Definition of Done**.

Extensión "SGEP 2026" (*Scrum Guide Expansion Pack*, Jocham/Coleman/Sutherland, 2026) [Fuente: Externa; diagramas de la cátedra generados con IA, síntesis visual, no el documento original]: - **Ciclo OODA** (Observar-Orientar-Decidir-Actuar) como lente adicional del empirismo. - **Rol "Supporter"**: líderes/gerentes que eliminan barreras organizacionales — un manager que no empodera al equipo no es Supporter, aunque tenga el título. - **IA como "compañero de equipo"**: puede actuar como Product Developer (automatizar generación de incrementos), pero *human in the loop*: la responsabilidad final es siempre 100% humana. - **Pensamiento de producto vs. proyecto**: los proyectos terminan, los productos son activos vivos ("desde la cuna hasta la tumba"). Product Vision = norte a largo plazo; Product Goal = corte vertical tangible; Product Backlog = lista emergente, nunca un plan fijo. - **Doble definición de "Hecho"**: **Definition of Output Done** (calidad técnica del incremento: ¿lo construimos correctamente?) vs. **Definition of Outcome Done** (valor de negocio real: ¿construimos lo correcto?, medido a menudo después del lanzamiento vía evidencia de comportamiento/ROI). El Output es requisito previo; el Outcome es el éxito real. - **Definition of Ready**: cualidad *emergente* del refinamiento continuo, no un compromiso formal — reduce riesgo antes de construir, sin sobre-refinar.

Planificación en múltiples niveles: Estrategia → Portfolio → Producto → Release → Iteración (Sprint) → Día — cada nivel resuelve preguntas más granulares. La **planificación de release** toma como insumo la visión, el Product Backlog, el roadmap y la **velocidad** del equipo, negociando alcance/fecha/presupuesto (misma lógica de la Restricción Triple aplicada a nivel release). Un release puede entregarse luego de varios sprints, de cada sprint, o de cada feature — decisión de negocio, no regla de Scrum. Ver [[Planificación de Release y Sprint]]. **Capacidad del equipo en un Sprint**: no es simplemente "todas las horas disponibles" — hay que descontar buffer del sprint, tiempo fuera del personal, compromisos externos, y las propias ceremonias del Sprint. Fórmula: **Horas de trabajo disponibles por día (WH) × Días disponibles (DA) = Capacidad**.

Herramientas: ambiente de trabajo abierto (silencio absoluto = mal signo), pizarrones, horarios definidos por el equipo. **Taskboard**: columnas Story/To Do/In Process/To Verify/Done; cada tarjeta lleva código de ítem, día del sprint, actividad, iniciales, horas estimadas. **Granularidad de tareas**: muchas tareas chicas y específicas = buen signo (progreso visible); una sola tarea gigante por historia = mala señal.

Gráficos de seguimiento: **Sprint Burndown** (trabajo restante día a día vs. línea ideal — por encima = atrasado, por debajo = adelantado); **Release Burnup** (línea "Done" acercándose a "Scope" a través de sprints — también muestra si el alcance cambió); **Product Burndown** (misma lógica a nivel de todo el backlog, con velocidad real sprint a sprint).

Métricas ágiles: "no todo lo que se puede contar cuenta, y no todo lo que cuenta se puede contar" (atribuido a Einstein). Regla de oro: la medición es una **salida**, no una actividad — medir lo necesario y nada más. Dos principios ágiles que guían qué medir: "satisfacer al cliente con entregas tempranas y continuas de software valioso" y "el software funcionando es la principal medida de progreso". **Running Tested Features (RTF)**: cuántas funcionalidades están terminadas *y* probadas por iteración (puede bajar si se rompe algo).

Cierre — ¿dónde está la agilidad?: la agilidad no vive en las prácticas concretas (TDD, daily, pairing — comportamientos visibles), sino en los principios (modelos mentales: calidad, simplicidad, colaboración) y, en la base, en los valores (creencias: individuos e interacciones, paciencia, humildad, valentía, adaptación,

inspección, feedback, transparencia, respeto). Un equipo puede hacer todas las ceremonias al pie de la letra y seguir sin ser ágil, si no sostiene esos valores.

UNIDAD 3 — SCM (Gestión de Configuración de Software)

20. Por qué hace falta gestionar la configuración

Un proyecto de software involucra personas que se incorporan a un proceso; ese proceso se adapta al proyecto y suele estar automatizado con herramientas; el resultado es un producto [Fuente: Apuntes]. Ejemplo de la clase: en enero, un árbol "Requerimientos → Diseño → Fuentes → Objetos" define una versión coherente. En febrero cambian algunas fuentes, generando nuevos objetos y, en consecuencia, nuevo diseño/requerimientos — el problema aparece cuando ese cambio no se propaga prolijamente (dos fuentes distintas terminan apuntando al mismo objeto, sin saber cuál "Febrero" es el correcto). Eso es exactamente lo que SCM existe para evitar.

Los cambios en el software se originan en: cambios del negocio y nuevos requerimientos, soporte de cambios de productos asociados, reorganización de prioridades, cambios de presupuesto, defectos a corregir, oportunidades de mejora [Fuente: Apuntes]. Sin disciplina de control: pérdida de componentes, pérdida de cambios, falta de sincronía fuente/objeto/ejecutable, regresión de fallas, doble mantenimiento, superposición de cambios, cambios no validados.

SCM es una actividad "paraguas": transversal a todo el proyecto, relevante para el producto a lo largo de todo su ciclo de vida — convive con Control de Calidad de Proceso, Control de Calidad de Producto y Prueba de Software como disciplinas de soporte dentro del Aseguramiento de Calidad [Fuente: Apuntes].

21. Definición, historia e integridad del producto

Origen a mediados de los 50, para hardware/control de producción, luego aplicado a software [Fuente: Apuntes]. Definición formal (ANSI/IEEE 828, 1990): *"una disciplina que aplica dirección y monitoreo administrativo y técnico a: identificar y documentar las características funcionales y técnicas de los ítems de configuración, controlar los cambios de esas características, registrar y reportar los cambios y su estado de implementación, y verificar correspondencia con los requerimientos"* [Fuente: Apuntes; coincide casi textual con Bersoff, "Elements of Software Configuration Management", *IEEE Trans. on SE*, 1984].

Bersoff agrega el objetivo detrás de la disciplina: la **"integridad del producto"** — que el software satisfaga las necesidades del usuario, sea rastreable en su ciclo de vida, cumpla criterios de performance, y cumpla expectativas de costo y entrega [Fuente: Externa — Bersoff, 1984, p. 430]. Bersoff también ubica a SCM dentro de un "triángulo de disciplinas": Management (general + de proyecto), Development (análisis, diseño, ingeniería, producción/codificación, prueba, instalación, documentación, entrenamiento) y **Product Assurance** — que incluye Quality Assurance, Test & Evaluation, Verification & Validation, y Configuration Management [Fuente: Externa — Bersoff, p. 431].

22. Conceptos clave

- **[[Ítem de Configuración]]**: cualquier artefacto del proyecto que puede cambiar, necesita compartirse, y del que hace falta conocer estado y evolución. Bersoff lo llama *Software Configuration Item (SCI)*, organizado en jerarquía de árbol; los de nivel más bajo aún no controlados por SCM se llaman *design objects* [Fuente: Externa — Bersoff, p. 433].
- **[[Repositorio de Control de Versiones|Repositorio]]**: almacén de ítems con su historial — centralizado (SVN) o descentralizado (Git).
- **[[Línea Base]]**: configuración de ítems con versión específica, revisada formalmente, que solo cambia por un procedimiento formal.
- **Versión**: forma particular de un artefacto en un momento dado — evolución lineal (1→2→3→4).
- **Variante**: versión que evoluciona por separado, configuración alternativa (ej. por plataforma) — se representa como árbol que se bifurca.
- **Ramas (branches)**: rama principal (trunk/master) + bifurcaciones con semántica propia; **merge** para integrar, con posibles conflictos resueltos comparando diffs.

23. Las cuatro actividades fundamentales de SCM

Coinciden exactamente con los 4 elementos de Bersoff: identificación, control, auditoría y contabilidad de estado [Fuente: Apuntes; Fuente: Externa — Bersoff, 1984].

a) Identificación de ítems: identificación unívoca, convenciones de nombrado, estructura del repositorio. Bersoff: tomar “instantáneas” (snapshots) — cada línea base y sus actualizaciones representan la evolución del software (Fig. 4 de Bersoff: “SCI evolution in a single document”, como fotogramas de una película).

b) Control de Cambios: propuesta de cambio → análisis de impacto → revisión → aceptación/rechazo → notificación. Decide el **Comité de Control de Cambios (CCC/CCB)**. Bersoff aclara: **la incorporación efectiva del cambio (escribir el código) NO es función de SCM** — SCM controla, evalúa y monitorea el proceso, no ejecuta el cambio en sí [Fuente: Externa — Bersoff, p. 434-435]. El documento formal es el **Engineering Change Proposal (ECP)**.

c) Auditorías de Configuración: dos tipos — **Auditoría física (PCA)** (asegura que lo indicado para el ítem se haya alcanzado realmente) y **Auditoría funcional (FCA)** (evalúa que la funcionalidad/performance real sea consistente con la especificación) [Fuente: Apuntes — estos nombres/siglas específicos vienen de la presentación de la cátedra, no aparecen textualmente en Bersoff, que habla de “software configuration auditing” en general sirviendo a dos propósitos: verificación de configuración y validación de configuración]. Relación con **V&V**: **Validación** (que el usuario obtenga el producto correcto) y **Verificación** (que el producto cumpla los objetivos preestablecidos en las líneas base). Bersoff: la auditoría aporta trazabilidad entre cada producto del ciclo de vida y los requerimientos que le dieron origen, haciendo visibles discrepancias (requerimientos no satisfechos, o funcionalidades sin requerimiento que las respalde) [Fuente: Externa — Bersoff, p. 436].

Flujo típico de una auditoría [Fuente: Apuntes]: Objetivo → Alcance → Preguntas → Checklist → Informe → Seguimiento. Roles: auditor (idealmente independiente), responsable del ítem, y el CCB (recibe el informe y decide qué hacer con las discrepancias).

d) Informes / Registro de Estado: mantiene los registros de evolución del sistema — suele automatizarse. Preguntas que responde: ¿cuál es el estado del ítem?, ¿un cambio fue aprobado o rechazado?, ¿qué versión implementa un cambio aprobado?, ¿cuál es la diferencia entre versiones? Bersoff detalla 9 tipos de datos que registra la contabilidad de estado (tiempos de baseline, tiempos de cada SCI, estado de ECPs, descripciones, deficiencias encontradas en auditorías, etc.) [Fuente: Externa — Bersoff, p. 437].

24. Plan de Gestión de Configuración

Debería incluir: reglas de nombrado, herramientas a usar, roles/integrantes del CCB, procedimiento formal de cambios, plantillas de formularios, procesos de auditoría [Fuente: Apuntes]. El *Little Book of Configuration Management* agrega: relación de CM con otras áreas (calidad, ingeniería, cliente, gerencia), evolución del proyecto en hitos/líneas base formales, gestión de interfaces, detalle de qué se registra en contabilidad de estado, gestión de subcontratistas [Fuente: Externa — *Little Book of Configuration Management*, 1998].

25. SCM en ambientes ágiles

Sin dejar de cumplir las 4 actividades fundamentales, SCM ágil [Fuente: Apuntes]: sirve a los practicantes y no al revés; hace seguimiento y coordina en vez de controlar a los desarrolladores; responde a los cambios en vez de evitarlos; es transparente y "sin fricción" (automatiza todo lo posible); prioriza coordinación y automatización frecuente; elimina desperdicio; prioriza documentación lean y trazabilidad sobre documentación extensa; da feedback continuo y visible. Es responsabilidad de todo el equipo, no de un rol aislado, y sus tareas quedan embebidas en el trabajo del sprint, no como etapa separada al final.

Los tres niveles de build [Fuente: Externa — Berczuk, Appleton, Konieczka, "Agile SCM", 2003]: **Build privado** (un dev, antes de subir cambios — mini auditoría personal), **Build de integración** (todo el equipo, diario — reemplaza al CCB presencial por verificación automática frecuente), **Build de release** (empaquetado formal para el cliente — sí incluye reportes automatizados de CM, como bill-of-materials).

Extensión moderna verificada (Quinteros/Guiñazu, terminología estándar de industria): la evolución práctica de esta idea son los pipelines de **Integración Continua (CI)**, **Entrega Continua (CD)** y **Despliegue Continuo**: Construcción → Pruebas unitarias → Despliegue → Pruebas de aceptación → Despliegue a producción, con grado de automatización creciente en cada una. Dos técnicas de despliegue asociadas: **Canary Deployment** (lanzamiento por etapas a un subconjunto de usuarios, para detectar problemas antes de exponer a todos) y **Blue-Green Deployment** (transferencia gradual de tráfico entre la versión anterior — "blue" — y la nueva — "green" — permitiendo revertir rápido si algo falla).

Advertencia coincidente en varios resúmenes de compañeros (no verificada contra un paper específico, pero consistente con el argumento de Berczuk sobre por qué el CCB presencial no aplica en ágil): de las 4 actividades de SCM, las **auditorías formales** son la que menos encaja con un contexto ágil puro, ya que ser auditado formalmente por alguien externo entra en tensión con la autogestión del equipo — en la práctica, esa función tiende a reemplazarse por las pruebas automatizadas de los pipelines de CI/CD.

Ventajas/desventajas de SCM en general (consistente entre varios resúmenes de compañeros): a favor — protege contra cambios innecesarios o no autorizados, da visibilidad y trazabilidad completa, fomenta auto-responsabilidad del equipo. En contra — puede generar quejas por burocracia percibida, la transición desde un equipo sin disciplina de SCM puede ser difícil, y sin compromiso real del equipo se vuelve papeleo vacío.

Nota final sobre fuentes y alcance

Todo el contenido de este documento corresponde a los temas del primer parcial (Unidad 1 completa + No Silver Bullet; Unidad 2: Gestión de Producto, Requerimientos Ágiles, User Stories, Estimaciones Ágiles, Scrum; Unidad 3: SCM). Se dejaron fuera, por no formar parte de este parcial, los temas de Testing, TDD, Lean/Kanban, PPQA, Revisiones Técnicas, Auditorías de Software, Retrospectivas y Métricas comparadas — todos con presentación propia en la cátedra, pero de unidades posteriores.

Las secciones marcadas `[Fuente: Externa]` fueron verificadas leyendo directamente el material original citado por la cátedra (Brooks, McConnell, Bersoff, Cohn, Ries, de Haaff), no tomadas de segunda mano de resúmenes de compañeros. Donde un resumen de compañero aportó contenido no verificable contra una fuente citada, se marcó explícitamente como tal (framework “Preparar un MVP”, advertencia sobre auditorías en ágil).